CSCI 12532
Computer Architecture and Design
Lecture 04

Ms. Meesha Mervyn
Dept. of Computer Systems Engineering
Faculty of Computing and Technology
University of Kelaniya
meesham@kln.ac.lk

CONTENTS

1. The Arithmetic and Logic Unit
   a. Introduction to ALU
   b. Importance in computer architecture and digital circuit design
   c. History of ALU
   d. Arithmetic Logic Unit
   e. Arithmetic Unit
   f. Logic Unit
2. Arithmetic Control Systems
3. Shift Control Systems
4. Logic Control Systems

1. INTRODUCTION TO ALU

An arithmetic logic unit (ALU) is a part of a central processing unit (CPU) that carries out arithmetic and logic operations. The ALU takes the input operands and an instruction and outputs the result.

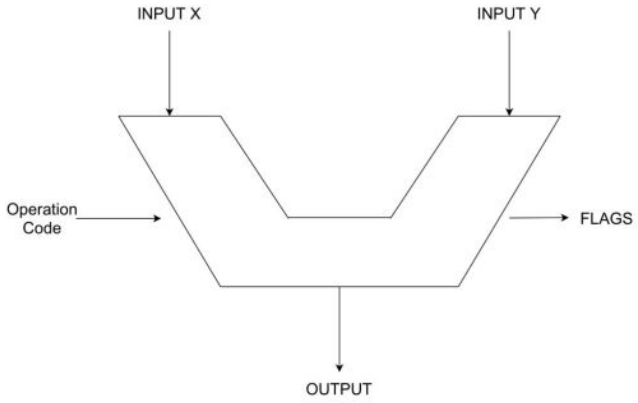
In some processors, the ALU is divided into two units: an arithmetic unit (AU) and a logic unit (LU). Some processors contain more than one AU -- for example, one for fixed-point operations and another for floating-point operations.

What is the purpose of an arithmetic logic unit?

The purpose of an ALU is to speed up a CPU's overall processing by performing math and logic functions. By splitting out these functions, the different portions of the CPU can be more specialized and perform different operations simultaneously.

All modern processors have an ALU or equivalent circuitry in the CPU. This includes x86, Arm and RISC-V CPU architectures. GPUs also contain circuitry to perform arithmetic, but these are more specialized than the ones in a CPU and are often called a different name.

Block Diagram of ALU



Importance in computer architecture and digital circuit design

1.Computation:

Arithmetic control systems enable mathematical operations like addition, subtraction, multiplication, and division.
These operations are critical for:

- Scientific calculations.
- Financial analysis.
- Graphics processing.

2.Data Manipulation:

Shift control systems facilitate the movement of bits within binary data.
They support operations such as Logical and arithmetic shifts & Circular shifts.
These operations are essential for tasks like:

- Data alignment.
- Extracting specific bits.
- Encoding/decoding operations.

3. Logical Operations:

Logic control systems manipulate logical values (true/false) using operations like AND, OR, XOR, and NOT.
These operations are the foundation of Boolean algebra.
They are vital for:

- Decision-making.
- Data filtering.
- Error detection.
- Control flow in programs.

4. Circuit Design:

Understanding these systems is crucial for designing efficient digital circuits.
Incorporating arithmetic, shift, and logic control systems allows engineers to create optimized hardware solutions for:

- Microprocessors.
- Digital signal processors.
- Memory systems.

History of ALU

ALUs originated in the early days of electronic computers.
Initially implemented using vacuum tubes and discrete electronic components.
Could only perform basic arithmetic operations like addition and subtraction.

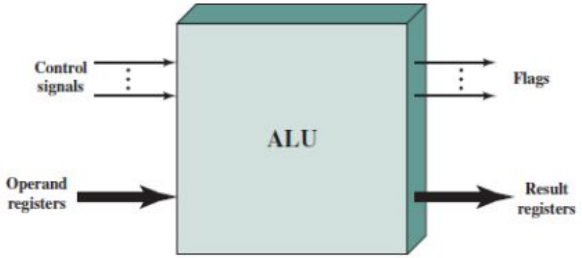
Key Milestones:

- 1945: Mathematician John von Neumann proposed the ALU concept in his report on the EDVAC computer.
- 1950s: Introduction of transistors led to smaller, more efficient ALUs.
- 1960s: Integrated circuits further miniaturized and enhanced ALU capabilities.
- 1970s: ALUs were integrated into microprocessors, combining with other CPU components for higher performance and lower cost.

Interconnection with the Processor

- Operands for arithmetic and logic operations are presented to the ALU via registers.
- Results are stored in registers, which are temporary storage locations connected to the ALU by signal paths.
- Flags: The ALU sets flags (e.g., overflow flag) to indicate specific conditions (e.g., result exceeds register capacity). Flag values are stored in processor registers.
- Control Signals: The processor provides signals to control ALU operations & manage data movement into and out of the ALU.

ALU Interconnection



Registers:
• Present data to ALU.
• Store result of an operation.
• Temporary storage locations within the processor that are connected by signal paths to the ALU.
• ALU set flags – result of an operation store in registers

Control Unit:
• Provides signal that control;
• Operation of the ALU
• Movement of the data into and out of the ALU.

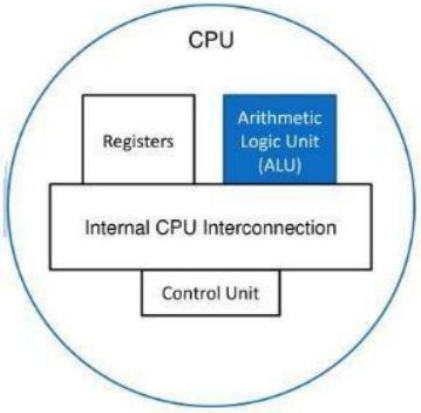


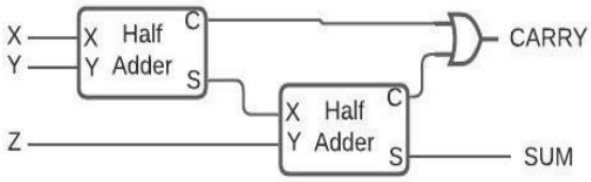
Fig 5 : ALU & Registers

Arithmetic Unit

Responsible for numerical operations like addition, subtraction, and increment operations.

Binary Addition Basics
Adding two binary digits:
0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 10 (results in 0 with a carry of 1).

For adding more than 1 + 1,
a full adder is required.



Quick Recall!

What are the different ways of representing signed values in computing?

Integer Representation in Computers

Binary Representation
Computers use 0s and 1s to represent information.
Two common methods for representing negative integers:

1. Sign Magnitude Representation.
2. Two's Complement Representation.

1. Sign Magnitude Representation

Uses the most significant bit (MSB) as the sign bit: 0 = Positive number, 1 = Negative number.
Example:
+18 = 00010010.
-18 = 10010010.

Drawbacks:

- Two representations of 0:
  +0 = 00000000.
  -0 = 10000000.
- Complex addition/subtraction: Requires considering both the signs and magnitudes of numbers.
- Rarely used in modern systems.

2. Two's Complement Representation:

MSB is the sign bit: 0 = Positive number, 1 = Negative number.
Positive Numbers: Represented as usual.
Example:
+3 = 00000011.
+2 = 00000010.
+1 = 00000001.
+0 = 00000000.

Negative Numbers: Represented by inverting bits and adding 1. (One's Complement + 1)
Example:
-1 = 11111111.
-2 = 11111110.
-3 = 11111101.

Advantages:

- Single representation of 0: 00000000.
- Simplifies addition/subtraction operations.
- Widely used in modern computers.

Logic Unit

Handles logical operations (e.g., AND, OR, XOR, NOT) instead of arithmetic operations.
Performs numerical tests and conditional checks on data.

Key Operations
Logical Operations:

- AND: Outputs 1 only if both input bits are 1.
- OR: Outputs 1 if at least one input bit is 1.
- XOR: Outputs 1 if the input bits are different.
- NOT: Inverts the input bit (1 becomes 0, 0 becomes 1).

Numerical Tests:

- Checks if a number is negative (by examining the sign bit).
- Determines if the output of the ALU is zero (zero flag).

Control Functions:

- Manages conditional logic for decision-making in programs.
- Sets status flags (e.g., zero flag, negative flag) based on the results of operations.

Use Cases - Data Filtering, Error Detection, Conditional Execution

2. Arithmetic Control Systems

A set of instructions and circuits designed to perform arithmetic operations in a computer system.
Involves manipulating numerical data (e.g., integers, floating-point numbers) to perform calculations and solve mathematical problems.

Enable computers to perform addition, subtraction, multiplication, division, and other arithmetic operations accurately and efficiently.

Essential for applications such as:
- Scientific simulations.
- Financial analysis.
- Data processing

Examples of Arithmetic Control Instructions

1.Addition:
Adds two operands to produce a sum.
Example: ADD A, B adds contents of registers A and B, storing the result in another register or memory location.

2.Subtraction:
Subtracts one operand from another to produce a difference.
Example: SUB A, B subtracts contents of register B from A, storing the result.

3.Multiplication:
Multiplies two operands to produce a product.
Example: MUL A, B multiplies contents of registers A and B, storing the result.

4.Division:
Divides one operand by another to produce a quotient and possibly a remainder.
Example: DIV A, B divides contents of register A by B, storing the quotient and remainder separately.

Circuit Implementation and Operation

Arithmetic control systems are implemented using Arithmetic Logic Units (ALUs) and associated circuits.

Operation:
- Inputs are taken from registers or memory.
- Data is processed using arithmetic circuits to produce results.

Control signals coordinate:
- Flow of data.
- Activation of appropriate arithmetic circuitry.
- Selection of operands.
- Direction of results to the desired destination (register or memory).

Overall Importance of Arithmetic control systems;
- Provides the computational power needed for various applications.
- Enables computers to perform complex mathematical calculations efficiently.
- Contributes to the functionality and versatility of digital systems.

- Imagine you have a calculator. The buttons on the calculator allow you to input numbers and perform various arithmetic operations. When you press the addition button, for example, the calculator's arithmetic control system kicks in. It takes the numbers you entered, adds them together, and displays the result.
- In more complex systems, such as computers or electronic devices, arithmetic control systems are responsible for executing mathematical operations behind the scenes. They work with binary numbers (0s and 1s) and use circuits and algorithms to perform calculations.
- These systems consist of components like arithmetic logic units (ALUs), registers, and control circuits. The ALU is the core component that performs arithmetic operations, such as adding or multiplying numbers. The registers store temporary data during calculations, and the control circuits coordinate and control the flow of data within the system.
- Arithmetic control systems are used in various applications, from basic calculators to sophisticated computers, where they handle complex calculations required for tasks like data processing, simulations, graphics rendering, and more.

3. Shift Control Systems

A set of instructions and circuits used to shift the position of bits within a binary representation of data.
Enables the movement of bits left or right for:

- Data manipulation.
- Extraction of specific bits.
- Data alignment.

Purpose is to provides flexibility in handling binary data for tasks like:

- Filling or emptying bit spaces.
- Extracting specific bit patterns.
- Aligning data for efficient processing.

Examples of Shift Control Instructions

Logical Shift
Moves bits left or right without considering the sign or value of the number.
Fills empty bit positions with zeros.

Logical Shift Left(LSL):
Shifts bits to the left.
Example: LSL A, 2 shifts contents of register A two positions left, filling with zeros.

Logical Shift Right(LSR):
Shifts bits to the right.
Example: LSR B, 3 shifts contents of register B three positions right, filling with zeros.

Arithmetic Shift
Shifts bits while preserving the sign bit (used for signed numbers).

Arithmetic Shift Left(ASL):
Shifts bits to the left, preserving the sign bit.
Example: ASL A, 1 shifts contents of register A one position left, preserving the sign bit.

Arithmetic Shift Right(ASR):
Shifts bits to the right, preserving the sign bit.
Example: ASR B, 2 shifts contents of register B two positions right, preserving the sign bit.

Key Applications
Data alignment for efficient processing.
Bit extraction for specific patterns.
Multiplication or division by powers of 2 (using left or right shifts).
Sign preservation in signed number operations.

Circular Shift
Rotates the bits in a binary number, creating a circular movement.
Bits that fall off one end are moved to the opposite end.

Circular Right Shift (ROR):
Shifts bits to the right.
Bits that fall off the right end are moved to the left end.
Example: ROR B, 4 rotates the contents of register B 4 positions to the right.

Key Feature:
No bits are lost; they are recycled to the other end.
Useful for bit rotation tasks in cryptography, data encoding

Let's use an 8-bit number to show the difference between Logical Shift Right (LSR) and Arithmetic Shift Right (ASR). We'll use the number 112 and its negative counterpart -112 as examples.

1. Shifting the Number 112
Bit pattern for 112 : 0111 0000
Shift : 3 places to the right.

Logical Shift Right (LSR):
Shifts bits to the right and fills empty spaces with 0s.

0111 0000 ➜ 0000 1110 (which is 14 in decimal).
This is equivalent to $112 / 2^3 = 14$.

Arithmetic Shift Right (ASR):
For positive numbers, ASR works the same as LSR.

0111 0000 ➜ 0000 1110 (which is 14 in decimal).

2. Shifting the Number -112
To represent -112 in 8-bit binary, we use 2's complement:
Start with the bit pattern for 112: 0111 0000.
Flip the bits: 1000 1111.
Add 1: 1000 1111 + 1 = 1001 0000.

Bit pattern for -112: 1001 0000

Logical Shift Right (LSR):
Shifts bits to the right and fills empty spaces with 0s.
1001 0000 ➜ 0001 0010 (which is +18 in decimal).

This result is incorrect for signed numbers because it doesn't preserve the sign.

Arithmetic Shift Right (ASR):
Shifts bits to the right and fills empty spaces with copies of the most significant bit (MSB).
Since the MSB is 1 (indicating a negative number), it fills with 1s.

1001 0000 ➜ 1111 0010 (which is -14 in decimal).

This is equivalent to $-112 / 2^3 = -14$, which is the correct result for signed numbers.

Difference Between Logical and Arithmetic Right Shifts

LSR: Always fills empty bits with 0s, regardless of the sign.

ASR: Preserves the sign by filling empty bits with the MSB (1 for negative, 0 for positive).

This makes ASR suitable for signed numbers, while LSR is used for unsigned numbers.

Circuit Implementation and Operation

Implementation
- Shift control systems are implemented using shift registers and associated logic circuits.
- Shift registers consist of a chain of flip-flops that hold binary data.
- Flip-flops are connected to allow data to shift from one flip-flop to the next.

Operation
- Control signals determine the direction and amount of shifting.
- These signals activate the appropriate circuitry within the shift register, causing bits to move as specified.
- Shifted data can be:
  - Stored back in the register.
  - Transferred to another location.

Applications
- Data manipulation: Rearranging and extracting specific bit patterns.
- Bit-level operations: Logical operations, bit masking, etc.
- Data encoding/decoding: Efficient processing of binary data.

Example of Binary Data Shifting in Digital Audio and Video Processing

Binary data shifting is widely used in compression and decompression algorithms for digital audio and video. These techniques help reduce file sizes while keeping the quality acceptable.

How It Works in Audio Compression?

- Audio Data Representation:
  Audio data is stored as a stream of binary values, representing the amplitude (intensity) of the sound at different points in time.

- Quantization:
  The audio data is split into small segments or frames, each containing binary values for a specific time interval.

- Bit-Rate Reduction:
  To reduce file size, the binary values in each frame are processed using binary shifting:
  Logical Shift Right:
  - ❏ Bits are shifted to the right, and empty spaces are filled with zeros.
  - ❏ This discards the least significant bits (LSBs), reducing precision but saving space.

  Arithmetic Shift Right:
  - ❏ Bits are shifted to the right, and empty spaces are filled with copies of the most significant bit (MSB).
  - ❏ This preserves the sign of the data while reducing precision.

- Reconstruction
  After compression, the audio data uses fewer bits per sample, making the file smaller.
  During playback or decompression:
  - ○ The shifted binary values are expanded back to their original bit depth.
  - ○ Discarded bits are restored using interpolation or other techniques.

Why Is It Useful?

- Efficient Storage and Transmission: Shifting reduces the number of bits needed to represent audio or video, making files smaller.
- Maintained Quality: By carefully discarding less important bits, the compressed file retains acceptable quality.

Real-World Applications

- Audio Compression: Formats like MP3 use shifting to reduce file sizes.
- Video Compression: Formats like MPEG apply similar techniques for video files.

4. Logic Control Systems

A set of instructions and circuits designed to perform logical operations on binary data.
Manipulates logical values (true/false) of individual bits or sets of bits.

Purpose is to provides the ability to perform Boolean operations like AND, OR, XOR, and NOT, which are essential for:

- Logical reasoning.
- Data filtering.
- Error detection.
- Program control flow.

Examples of Logic Control Instructions

- AND Operation:
  Performs a logical AND between two operands, bit by bit.
  Result bit is 1 only if both corresponding bits in the operands are 1.
  Example: AND A, B performs AND between contents of registers A and B.

- OR Operation:
  Performs a logical OR between two operands, bit by bit.
  Result bit is 1 if at least one corresponding bit in the operands is 1.
  Example: OR A, B performs OR between contents of registers A and B.

- XOR Operation:
  Performs a logical XOR between two operands, bit by bit.
  Result bit is 1 only if the corresponding bits in the operands are different (one is 1, the other is 0).
  Example: XOR A, B performs XOR between contents of registers A and B.

- NOT Operation:
  Performs a logical negation on a single operand, bit by bit.
  Inverts (flips) each bit (1 becomes 0, 0 becomes 1).
  Example: NOT A performs negation on the contents of register A.

Circuit Implementation and Operation

Implementation:
- Logic control systems are implemented using logic gates:
- AND gates, OR gates, XOR gates, and NOT gates.
- These gates process binary inputs and produce binary outputs based on the logical operation.

Operation:
- Control signals direct the flow of data through the logic gates.
- Signals activate the appropriate gates and determine connections between inputs and outputs.

Applications:
- Decision-making: Enables computers to make decisions based on conditions.
- Data filtering: Filters data based on logical conditions.
- Error detection: Detects errors in data transmission or storage.
- Control flow: Controls the execution of instructions in programs.

Example of a Home Security System

Inputs:
- Binary signals from sensors (e.g., motion detectors, door/window sensors, cameras).
- Signals indicate 0 (no breach) or 1 (potential breach).

Logic Operations:
- AND Gate: Checks if multiple sensors are triggered simultaneously.
  Example: Confirms a breach if both motion and door sensors are activated.

- OR Gate: Checks if any sensor detects a breach.
  Example: Triggers an alarm if any sensor is activated.

Outputs:
- Alarm Activation: Triggers an alarm if a breach is confirmed.
- Camera Activation: Activates security cameras to record suspicious activity.
- Door Locks: Integrates with automated locks to lock/unlock based on conditions.

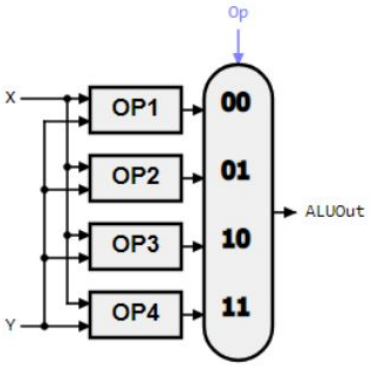
Building a 1-bit ALU

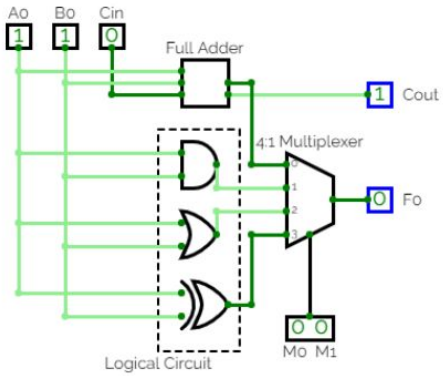
Let's construct a simple ALU that performs a arithmetic operation (1 bit addition) and does 3 logical operations namely AND, OR and XOR as shown below. The multiplexer selects only one operation at a time. The operation selected depends on the selection lines of the multiplexer as shown in the truth table.
Input = M0,M1 & Output = Operation

M0	M1	Operation
0	0	SUM
0	1	AND
1	0	OR
1	1	XOR

Steps

1. Draw the block diagram
2. Open logisim simulator and simulate it.





MULTIPLEXER (MUX)

- A Multiplexer is a device that allows one of several analog or digital input signals which are to be selected and transmits the input that is selected into a single medium. Multiplexer is also known as Data Selector.
- A multiplexer of 2n inputs has n select lines that will be used to select input line to send to the output. Multiplexer is abbreviated as Mu

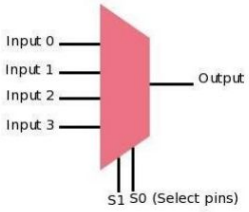


Fig 11 : Block diagram of 4:1 Mux

THANK YOU!

20.03.2025